Agentic Research Assistant: Reference Architecture

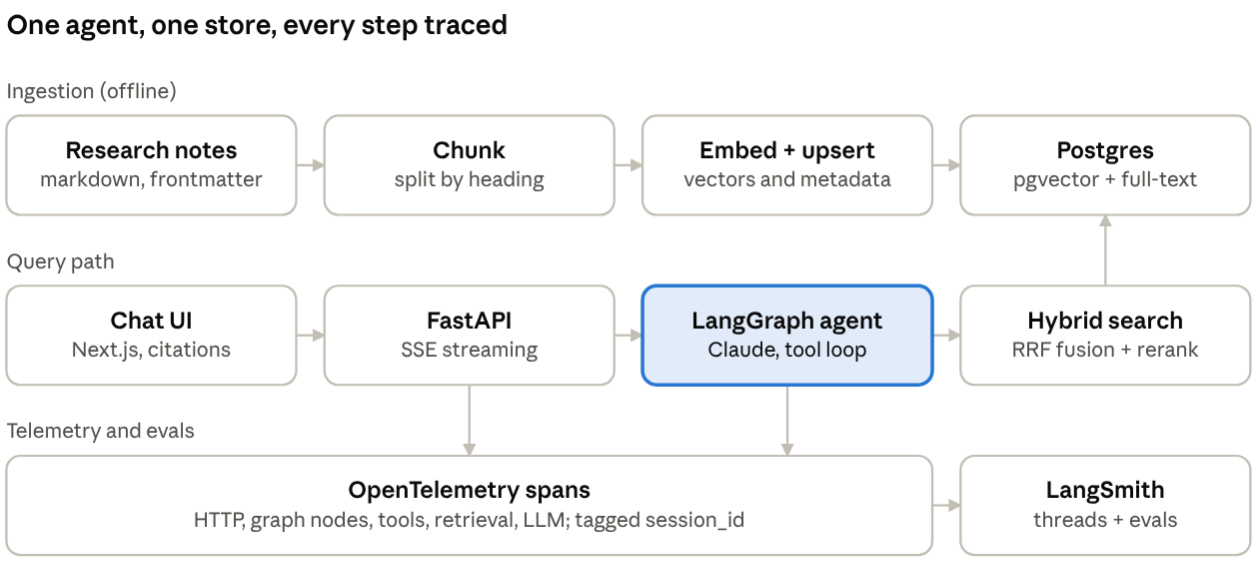
Oct 19, 2026 · @Haydn Murray

Recommendation

Build one LangGraph agent behind a FastAPI streaming API, over Postgres + pgvector with hybrid search and a reranker, with a thin Next.js chat UI, all running from `docker compose up`. Trace everything through OpenTelemetry and export to LangSmith; run LangSmith evals in CI as a release gate.

This keeps the parts few enough to explain on a whiteboard while still showing what interviewers probe: retrieval quality, grounding, observability and evaluation.

Flow



architecture · ingestion, query path, telemetry

Notes are chunked and embedded offline; at query time the agent calls hybrid search over the same Postgres store, and every step emits OTel spans tagged with `session_id` so LangSmith groups each conversation as a thread.

Stack

Layer	Choice	Why	Alternative
Agent	LangGraph	Familiar, native LangSmith fit	Plain tool-use loop for zero framework
LLM	Claude (Anthropic API)	Strong tool use and citation behaviour	Bedrock to look AWS-shaped
Store	Postgres + pgvector	Hybrid search and SQL metadata filters in one container	Qdrant or OpenSearch at real scale
Rerank	Cohere Rerank or local cross-encoder	Cheapest big win in retrieval quality	Skip only if latency matters more
API	FastAPI + SSE	Simple streaming, easy to instrument	
UI	Next.js + Vercel AI SDK, citations panel	Looks real, little code	Streamlit for ~50 lines
Tracing	OTel SDK to LangSmith OTLP endpoint	Vendor-neutral; same traces could go to Phoenix or Datadog	LangSmith SDK native tracing
Evals	LangSmith datasets + `evaluate()` in GitHub Actions	Mirrors a production release-gate story	

Make "agentic" earn its place

An agent that just calls `search` once is RAG with extra latency. Give it tools that justify the loop:

- `search_notes(query, ticker?, sector?, date_from?, date_to?)`, so it extracts filters from the question
- `get_note(note_id)`, to pull a full note when a chunk isn't enough
- Questions that need multi-hop, e.g. "how has our view on ASML changed since Q1?" (find notes over time, compare ratings)

Evals

Seed with ~50 synthetic notes and ~30 graded questions, scored at four levels.

Level	What it checks	How
Retrieval	recall@k and MRR against gold note IDs	Deterministic, cheap; catches most regressions
Answer	Correctness vs reference; groundedness (every claim supported by a cited chunk)	LLM-as-judge
Trajectory	Right filters applied, right tools called	Compare tool calls to expected trajectory
Online	User thumbs up/down	Written as LangSmith feedback on the trace

MVP vs later

MVP

- [ ] Ingestion of markdown notes with frontmatter
- [ ] Hybrid retrieval + rerank
- [ ] Agent with two tools
- [ ] Chat UI with citations
- [ ] OTel traces to LangSmith, grouped by session
- [ ] Offline eval suite gating CI

Later

- Entitlement filtering per user: research distribution and MNPI controls belong in the retrieval filter, not the prompt (a strong FS talking point)
- Online evals on production traces
- Query rewriting
- Semantic caching

Open decision

Model provider: Anthropic API or Bedrock. Default to the Anthropic API for an interview demo (one key, runs anywhere); Bedrock looks closer to an AWS production stack.